

Överlagring och friends

I normal vektoraritmetik kan man multiplicera vektorer med skalärer, d.v.s. vanliga tal för att ändra vektorns längd. Man multiplicerar då varje komponent med talet. Vi överlagra `*` på följande sätt:

```
// prototyp
Vector operator* (const float S) const;

// implementation
Vector Vector::operator* (const float S) const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( m_X * S );
    Result.setY ( m_Y * S );
    Result.setZ ( m_Z * S );

    // returnera den nya vektorn
    return Result;
}
```

Vi kan nu skriva kod på t.ex. följande sätt:

```
#include <iostream>
#include "Vector5.h"

int main () {
    Vector V1 ( 1, 2, 3 );

    // multiplicera med skalär
    Vector Result = V1 * 3;

    cout << "x=" << Result.x () << ", "
         << "y=" << Result.y () << ", "
         << "z=" << Result.z () << endl;
}
```

Kör vi programmet ovan får vi följande resultat:

```
% ./TestVector5
x=3, y=6, z=9
%
```

Vi ser att varje komponent multiplicerats med tre, precis som vi avsåg. Men vad händer om vi istället skriver något i stil med följande:

```
Vector V1 ( 1, 2, 3 );

// multiplicera med skalär
Vector Result = 3.0 * V1;
```

Detta skulle innebära att datatypen `float` borde ha operatoren `*` överlagrad på ungefär följande sätt:

```
float.operator* (Vector);
```

Någon dylik funktion/metod finns inte. Vi märker alltså att det är den vänstra operanden som måste ha metoden överlagrad, och primitiva datatyper har inga överlagringar för icke-primitiva datatyper (såsom t.ex. `Vector` i vårt fall). Vi kan göra det enkelt för oss och säga att man måste ha skalärer till höger om en `Vector` då man utför multiplikation med skalär, men det är ointuitivt och leder till onödiga problem för programmeraren.

Lösningen är att använda en funktion som inte är medlem av klassen `Vector`.

Friend-funktioner

Eftersom det är omöjligt att definiera en metod för klassen `Vector` så att man kan multiplicera från vänster med en skalär skapar vi en funktion som är utanför klassen helt och hållet. Eftersom det är en vanlig funktion behöver man inget objekt för att anropa metoden, men istället tar den två parametrar, en `float` och en `Vector`. Vad vi gör är alltså att vi definierar om den generella operatoren `+` för argumenten `float` och `Vector`. Koden ser ut på följande sätt:

```
// prototyp
friend Vector operator* (const float S, const Vector & V);

// implementation
Vector operator* (const float S, const Vector & V) {
    // använd tidigare överlagrad operator '*'
    return V * S;
}
```

Notera att vi definierar *funktionen* som en *friend* till klassen `Vector` (för mera information om friends se [avsnittet Friends i Kapitel 17](#)). På det sättet kan funktionen accessera privata data i parametern `V` om så skulle behövas. Vi har dock gjort funktionen elegant och använder oss av den tidigare överlagrade operatoren `*` som tillhör klassen. För att kunna göra detta så reverserar vi ordningen på `S` och `V`, och får en multiplikation mellan en `Vector` och en `float`, vilket ju kan hanteras av den tidigare versionen.

Vi kan nu skriva t.ex. följande kod utan problem:

```
Vector V1 ( 1, 2, 3 );

// multiplicera med skalär
Vector Result = 3.0 * V1;
```

Ovanstående funktion blir om man använder sig av den oföskönade syntaxen följande:

```
Vector V1 ( 1, 2, 3 );

// multiplicera med skalär
Vector Result = operator* (3.0, V1);
```

Vi kunde om vi ville definiera alla överlagrade operatorer som funktioner på samma sätt som vi gjorde ovan, men det är inte speciellt vackert och bryter emot principen att kapsla ihop kod som har med en klass att göra med klassen själv. Viss funktionalitet måste dock implementeras via friend-funktioner.

[Förutgående](#)

Överlagring av operatörn –

[Hem](#)

[Upp](#)

[Nästa](#)

Operatorerna == och =